

Conferencia 3: funciones

Programación · Ciencia de la Computación y Ciencia de Datos · MatCom · 2026-09-30

Un número entero mayor que 1 es **primo** si no se puede dividir exactamente entre ningún otro salvo 1 y él mismo. Decidirlo para un número dado está al alcance de lo que ya sabes: un ciclo, un `if` y el operador `%`. Lo que no está al alcance todavía es usar ese programa dentro de otro, y por eso hoy aprendemos a ponerle nombre a un pedazo de programa.

Al final de la clase tendremos dos programas que imprimen todos los primos hasta un número dado. Uno los va probando de uno en uno, y el otro, que tiene dos mil trescientos años, los encuentra sin dividir ni una sola vez.

1. Un programa sin nombre

Empecemos por el problema de un solo número. ¿Es 91 primo? Con lo de la clase pasada se escribe así:

```
n = 91
primo = True

for d in range(2, n):
    if n % d == 0:
        primo = False

print(primo)
```

False

Es el patrón acumulador con un `bool` en lugar de un número: `primo` empieza en `True`, y cualquier divisor que aparezca lo tumba a `False`. El ciclo prueba todos los enteros desde 2 hasta $n - 1$, que son todos los candidatos posibles a divisor.

El programa está bien y sirve de poco. Para probar otro número hay que editar la primera línea y volver a correrlo. Y si quisieras los primos menores que cien, necesitarías meter este programa dentro de otro ciclo cien veces, y no hay manera de hacerlo: **el programa no tiene nombre, así que no se puede mencionar.**

Una función es exactamente eso, un nombre para un pedazo de programa. Se define con `def`, un nombre, unos paréntesis y dos puntos, y el cuerpo va indentado igual que el de un `if` o un `while`:

```
def es_primo(n):
    primo = True

    for d in range(2, n):
        if n % d == 0:
            primo = False

    return primo

print(es_primo(91))
print(es_primo(97))
```

False
True

Las mismas seis líneas de antes, y ahora `es_primo` es una palabra del vocabulario del programa. 91 es $7 * 13$, así que la primera respuesta es `False`; 97 sí es primo.

Hay dos momentos distintos aquí y conviene separarlos desde ahora. La **definición** es el bloque del `def`: le dice a Python qué significa ese nombre, y no ejecuta nada. La **llamada** es `es_primo(91)`, con los paréntesis: ahí es donde el cuerpo corre de verdad. Definir una función y no llamarla nunca es perfectamente legal:

```
def es_primo(n):  
    print("me llamaron")  
  
print("el programa terminó")
```

```
el programa terminó
```

Los paréntesis son obligatorios para llamar. Sin ellos no hay error, pero tampoco pasa nada útil, porque el nombre a secas se refiere a la función misma y no a su resultado:

```
def es_primo(n):  
    return n == 2  
  
print(es_primo)
```

```
<function es_primo at 0x7c523aa33530>
```

Eso que imprime es la función como valor. Hoy no lo vamos a usar, pero recuérdalo: significa que una función es un dato más, como un número o un texto, y sobre esa idea está construida media biblioteca estándar.

Antes de seguir, una advertencia sobre nuestra `es_primo`: dice que 1 es primo, y dice que 0 también. Los dos son mentira. Lo arreglamos en la sección 3.

2. Parámetros y argumentos

`n` es un **parámetro**: una variable que se escribe entre los paréntesis de la definición y que recibe un valor distinto en cada llamada. Puede haber varios, separados por comas, y se emparejan por posición con los valores de la llamada:

```
def es_divisible(n, d):  
    return n % d == 0  
  
print(es_divisible(91, 7))  
print(es_divisible(91, 5))
```

```
True  
False
```

Esa función chiquita nos va a servir después. De momento fíjate en dos palabras que se confunden todo el tiempo y que vas a oír en los exámenes. **Parámetro** es el nombre que aparece en la definición: `n`, `d`. **Argumento** es el valor concreto que se pasa en la llamada: 91, 7. El parámetro es el hueco y el argumento es lo que se le mete.

Como el emparejamiento es por posición, el orden importa y equivocarlo no da ningún error, solo una respuesta falsa:

```
def es_divisible(n, d):  
    return n % d == 0  
  
print(es_divisible(7, 91))
```

```
False
```

7 % 91 es 7, que no es cero, así que la respuesta es False. Lo que preguntamos sin darnos cuenta fue si 7 es divisible entre 91.

Si el número de argumentos no coincide con el de parámetros, ahí sí Python se queja, y se queja antes de ejecutar nada del cuerpo:

```
def es_divisible(n, d):  
    return n % d == 0  
  
print(es_divisible(91))
```

```
Traceback (most recent call last):  
  File "<cell>", line 4, in <module>  
    print(es_divisible(91))  
          ~~~~~^  
TypeError: es_divisible() missing 1 required positional argument: 'd'
```

3. return

return es la instrucción que entrega un valor a quien llamó. Es lo más útil que puede hacer una función, porque el que llama se queda con el resultado y hace con él lo que quiera:

```
def es_divisible(n, d):  
    return n % d == 0  
  
if es_divisible(100, 4) and es_divisible(100, 25):  
    print("100 se divide entre 4 y entre 25")
```

```
100 se divide entre 4 y entre 25
```

Recupera la distinción de la primera clase: **una función con return es una expresión**, y por eso se puede meter dentro de un and, de una suma o de otra llamada.

Imprimir no es devolver

La diferencia entre imprimir y devolver es la que más trabajo cuesta al principio, así que vamos a verla de frente. Estas dos funciones parecen hacer lo mismo:

```
def divisible_imprime(n, d):  
    print(n % d == 0)  
  
def divisible_devuelve(n, d):
```

```

    return n % d == 0

divisible_imprime(100, 4)
print(divisible_devuelve(100, 4))

```

```

True
True

```

La salida es idéntica. Pero intenta contar con ellas los divisores de 100:

```

def divisible_imprime(n, d):
    print(n % d == 0)

def divisible_devuelve(n, d):
    return n % d == 0

divisores = 0

for d in range(2, 11):
    if divisible_devuelve(100, d):
        divisores += 1

print(f"la que devuelve encontró {divisores} divisores")

print(divisible_imprime(100, 4) + divisible_imprime(100, 5))

```

```

la que devuelve encontró 4 divisores
True
True
Traceback (most recent call last):
  File "<cell>", line 15, in <module>
    print(divisible_imprime(100, 4) + divisible_imprime(100, 5))
    ~~~~~^~~~~~
TypeError: unsupported operand type(s) for +: 'NoneType' and 'NoneType'

```

La que imprime no sirve para calcular. Escribe en la pantalla y no deja nada. La regla práctica: **una función que calcula algo lo devuelve; imprimir es trabajo del programa principal**. Así puedes cambiar la presentación sin tocar el cálculo, y eso es justamente lo que vamos a hacer con `es_primo` en un rato.

Funciones que no devuelven nada

¿Qué valor tiene entonces `divisible_imprime(100, 4)`? El error de arriba lo dice: `None`, que es lo que Python devuelve cuando no le dices otra cosa. `None` es un valor de pleno derecho, con su propio tipo, y significa «nada»:

```

def divisible_imprime(n, d):
    print(n % d == 0)

resultado = divisible_imprime(100, 4)
print(resultado)
print(type(resultado))

```

```

True
None
<class 'NoneType'>

```

Una función sin return no está mal escrita. print misma es una de ellas. Lo que está mal es esperar un valor de una función que no devuelve ninguno.

return termina la función

En cuanto se ejecuta un return, la función se acaba y lo que venga después no corre. Eso arregla los dos problemas que dejamos abiertos:

```
def es_primo(n):
    if n < 2:
        return False

    for d in range(2, n):
        if n % d == 0:
            return False

    return True

print(es_primo(0), es_primo(1), es_primo(2), es_primo(91), es_primo(97))
```

```
False False True False True
```

Las tres líneas con return hacen tres trabajos distintos. La primera es un **filtro de entrada**: 0, 1 y los negativos no son primos, y la función lo dice y se va sin entrar al ciclo. La segunda está dentro del ciclo, y ahí está lo bueno: en el momento en que aparece un divisor ya no hay nada más que averiguar, así que la función contesta y sale. No hace falta break, porque return sale del ciclo y de la función a la vez. La tercera solo se alcanza si el ciclo terminó sin encontrar nada.

Compara esa versión con la de la sección 1. La de la sección 1 encuentra el divisor 7 de 91 y sigue probando hasta el 90, con la respuesta ya decidida. Cuánto cuesta esa terquedad se ve poniéndole el reloj. time.perf_counter() devuelve un número de segundos que solo sirve para restarlo de otro:

```
import time

def con_bandera(n):
    if n < 2:
        return False

    primo = True

    for d in range(2, n):
        if n % d == 0:
            primo = False

    return primo

def con_return(n):
    if n < 2:
        return False

    for d in range(2, n):
        if n % d == 0:
            return False

    return True

inicio = time.perf_counter()
print(con_bandera(1000000), f"{time.perf_counter() - inicio:.6f} s")
```

```
inicio = time.perf_counter()
print(con_return(1000000), f"{time.perf_counter() - inicio:.6f} s")
```

```
False 0.042972 s
False 0.000018 s
```

Un millón es par. La primera versión hizo casi un millón de divisiones para decirlo. La segunda hizo una.

4. Componer funciones

Con `es_primo` en el vocabulario, el programa que la clase pasada era imposible se escribe en cuatro líneas:

```
def es_primo(n):
    """Dice si n es primo, probándolo contra todos los enteros menores que él."""
    if n < 2:
        return False

    for d in range(2, n):
        if n % d == 0:
            return False

    return True

def primos_hasta(n):
    """Imprime todos los primos entre 2 y n, separados por espacios."""
    for k in range(2, n + 1):
        if es_primo(k):
            print(k, end=" ")

    print()

primos_hasta(100)
```

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
```

Lee `primos_hasta` y fíjate en lo que **no** dice. No dice nada de %, ni de divisores, ni de ciclos internos. Dice: recorre los números del 2 al `n`, y de cada uno pregunta si es primo. Eso es lo que compran las funciones. `primos_hasta` no sabe cómo se decide si un número es primo; sabe que hay alguien que lo decide.

El método se llama **descomponer**, y la pregunta que lo guía es siempre la misma: ¿puedo explicar lo que hace este pedazo en una sola frase? Si la respuesta es sí, ese pedazo es una función y esa frase es su nombre. «Dice si un número es primo» y «imprime los primos hasta `n`» son dos frases, así que son dos funciones.

Una tercera frase, que vamos a usar todo el resto de la clase: «cuenta cuántos primos hay hasta `n`».

```
def contar_primos(hasta):
    """Devuelve cuántos primos hay entre 2 y `hasta`."""
    total = 0

    for k in range(2, hasta + 1):
        if es_primo(k):
            total += 1

    return total
```

```
print(contar_primos(100), "primos hasta 100")
```

```
25 primos hasta 100
```

contar_primos es el mismo recorrido de primos_hasta con un acumulador en lugar de un print. Devuelve, no imprime, y por eso se puede cronometrar sin llenar la pantalla de números.

5. Mejorar en un solo lugar

es_primo es correcta y es tonta. Para saber si 999983 es primo prueba a dividirlo entre 999981 números, y casi todos son absurdos. Vamos a arreglarla tres veces, y lo que importa de las tres veces es **dónde** se hace el arreglo.

Primero, la medición de partida. medir es otra función de una frase: cuenta los primos hasta un tope y dice cuánto tardó.

```
import math
import time

def es_primo(n):
    if n < 2:
        return False

    for d in range(2, n):
        if n % d == 0:
            return False

    return True

def contar_primos(hasta):
    total = 0

    for k in range(2, hasta + 1):
        if es_primo(k):
            total += 1

    return total

def medir(hasta):
    """Cuenta los primos hasta `hasta` e imprime cuánto tardó."""
    inicio = time.perf_counter()
    total = contar_primos(hasta)
    print(f"{total} primos hasta {hasta} en {time.perf_counter() - inicio:.3f} s")

medir(10000)
```

```
1229 primos hasta 10000 en 0.249 s
```

Primera mejora. Ningún divisor de n , aparte de n mismo, puede ser mayor que $n // 2$. Probar más arriba es tiempo perdido:

```
def es_primo(n):
    if n < 2:
        return False

    for d in range(2, n // 2 + 1):
        if n % d == 0:
            return False
```

```
return True
```

```
medir(10000)
```

1229 primos hasta 10000 en 0.125 s

Segunda mejora. Si $n = a * b$ y los dos factores fueran mayores que $\sqrt{n}$, el producto pasaría de n . Así que en cualquier descomposición hay un factor que no llega a la raíz, y basta buscar ahí:

```
def es_primo(n):
    if n < 2:
        return False

    for d in range(2, int(math.sqrt(n)) + 1):
        if n % d == 0:
            return False

    return True

medir(10000)
```

1229 primos hasta 10000 en 0.006 s

Tercera mejora. Si n es par y no es 2, ya está resuelto. Y si no es par, ningún divisor suyo lo es, así que el ciclo puede ir de dos en dos:

```
def es_primo(n):
    if n < 2:
        return False

    if n == 2:
        return True

    if n % 2 == 0:
        return False

    for d in range(3, int(math.sqrt(n)) + 1, 2):
        if n % d == 0:
            return False

    return True

medir(10000)
```

1229 primos hasta 10000 en 0.004 s

De décimas de segundo a milésimas, sin tocar el ciclo de `medir` ni el de `contar_primos` ni el de `primos_hasta`. Los tres siguen diciendo `es_primo(k)`, la misma palabra de siempre, y los tres corrieron más rápido porque alguien cambió una línea en otro lugar.

Ese es el argumento entero de la clase de hoy, y es mucho más fuerte que «no repitas código». Una función es un contrato: dice qué recibe y qué devuelve, y no dice cómo. Todo lo que está dentro del contrato se puede cambiar sin avisarle a nadie. Un programa sin funciones no tiene contratos, y entonces cualquier mejora hay que aplicarla en todos los lugares donde se copió, y descubrir cuáles son es tu problema.

6. Alcance de las variables

Las variables que nacen dentro de una función viven solo ahí. Cuando la función termina, desaparecen, y eso incluye a la variable del ciclo:

```
import math

def primer_divisor(n):
    """Devuelve el menor divisor de n mayor que 1, o n mismo si es primo."""
    for d in range(2, int(math.sqrt(n)) + 1):
        if n % d == 0:
            return d

    return n

print(primer_divisor(91))
print(d)
```

```
7
Traceback (most recent call last):
  File "<cell>", line 12, in <module>
    print(d)
    ^
NameError: name 'd' is not defined. Did you mean: 'id'?
```

Eso se llama **alcance local**, y no es una limitación sino la razón de ser de las funciones. `primer_divisor` usa una `d` sin preguntarle permiso a nadie, y ninguna otra parte del programa puede estorbarla. Cada función es una caja cerrada.

Al revés sí funciona: desde dentro se puede **leer** una variable de afuera, llamada global.

```
TOPE = 100

def esta_en_rango(n):
    return 2 <= n <= TOPE

print(esta_en_rango(50))
print(esta_en_rango(500))
```

```
True
False
```

Pero si intentas **asignarle** un valor a una variable global desde dentro, pasa algo que sorprende a todo el mundo la primera vez. Digamos que queremos saber cuántas divisiones hace `es_primo`, y llevamos la cuenta en una variable de afuera:

```
import math

divisiones = 0

def es_primo(n):
    if n < 2:
        return False

    for d in range(2, int(math.sqrt(n)) + 1):
        divisiones = divisiones + 1
```

```

        if n % d == 0:
            return False

    return True

print(es_primo(97))

```

```

Traceback (most recent call last):
  File "<cell>", line 17, in <module>
    print(es_primo(97))
    ~~~~~^~~~~
  File "<cell>", line 10, in es_primo
    divisiones = divisiones + 1
    ^^^^^^^^^^^
UnboundLocalError: cannot access local variable 'divisiones' where it is not associated with a value

```

Python decide si una variable es local o global mirando el cuerpo entero de la función antes de ejecutarlo. Como ahí dentro hay una asignación a `divisiones`, la declara local para toda la función, incluida la línea que intenta leerla. Y esa variable local todavía no tiene valor cuando se la quiere leer.

Existe la palabra `global` para forzar el otro comportamiento. Casi nunca es la solución correcta, y en este curso no la vamos a usar. **Si una función necesita un valor de afuera, se lo pasas como parámetro; si produce un valor, lo devuelve con `return`.** Ese es el contrato de la sección anterior, y el contador de divisiones respeta el contrato así:

```

import math

def divisiones_para(n):
    """Devuelve cuántas divisiones hacen falta para decidir si n es primo."""
    cuenta = 0

    for d in range(2, int(math.sqrt(n)) + 1):
        cuenta += 1

        if n % d == 0:
            return cuenta

    return cuenta

print(divisiones_para(1000000), "divisiones para 1000000")
print(divisiones_para(999983), "divisiones para 999983")

```

```

1 divisiones para 1000000
998 divisiones para 999983

```

Un número par se descarta con una sola división. Un primo de seis cifras cuesta casi mil. Contar operaciones en lugar de medir segundos es una idea a la que le vamos a dedicar una conferencia entera.

7. Valores por defecto y argumentos con nombre

Un parámetro puede traer un valor de fábrica, que se usa cuando la llamada no lo menciona.

`contar_primos` cuenta desde 2, y si alguna vez queremos otro punto de partida no hace falta una segunda función:

```

import math

def es_primo(n):
    if n < 2:
        return False

    if n == 2:
        return True

    if n % 2 == 0:
        return False

    for d in range(3, int(math.sqrt(n)) + 1, 2):
        if n % d == 0:
            return False

    return True

def contar_primos(hasta, desde=2):
    """Devuelve cuántos primos hay entre `desde` y `hasta`."""
    total = 0

    for k in range(desde, hasta + 1):
        if es_primo(k):
            total += 1

    return total

print(contar_primos(100))
print(contar_primos(100, 50))
print(contar_primos(100, desde=90))

```

```

25
10
1

```

Los parámetros con valor por defecto van siempre al final de la lista, porque si no Python no sabría cómo emparejar los argumentos por posición. Y en la llamada se puede nombrar el parámetro explícitamente, como en la última línea, en cuyo caso el orden ya no importa.

Esto sirve para dos cosas. Una función con buenos valores por defecto es cómoda en el caso corriente y sigue siendo flexible en el raro: la mayoría de las llamadas van a ser `contar_primos(100)`. Y una llamada como `contar_primos(100, desde=90)` se lee sin ir a mirar la definición, cosa que `contar_primos(100, 90)` no.

8. La criba de Eratóstenes

Todo lo de hoy comparte una idea: para saber si un número es primo, lo interrogamos. Hay otra manera, y es de las más antiguas que se conocen. En lugar de preguntar número por número, **se tacha**.

Escribe los números del 2 al 25 en una fila. El 2 es primo, y de un tirón tacha todos sus múltiplos. El siguiente sin tachar es el 3, que por eso mismo es primo, y tacha todos los suyos. Después el 5. Lo que queda sin tachar son los primos, y no se hizo ni una división:

```

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
2 3 . 5 . 7 . 9 . 11 . 13 . 15 . 17 . 19 . 21 . 23 . 25
2 3 . 5 . 7 . . . 11 . 13 . . . 17 . 19 . . . 23 . 25
2 3 . 5 . 7 . . . 11 . 13 . . . 17 . 19 . . . 23 . .

```

Eso es la **criba de Eratóstenes**, y para escribirla en Python hace falta un dato que no hemos visto: una fila de casillas donde anotar los tachones. Se llama **lista**, la clase que viene es entera sobre ellas, y hoy solo necesitas cuatro cosas:

SE ESCRIBE	QUÉ HACE
<code>[False] * (n + 1)</code>	una fila de $n + 1$ casillas, todas con <code>False</code>
<code>casillas[k]</code>	lee la casilla número k , contando desde cero
<code>casillas[k] = True</code>	escribe en la casilla número k
<code>primos.append(k)</code>	añade k al final de la lista <code>primos</code>

Con eso la criba cabe en doce líneas:

```
def criba(n):  
    """Devuelve la lista de los primos hasta n, por el método de Eratóstenes."""  
    compuesto = [False] * (n + 1)  
    primos = []  
  
    for candidato in range(2, n + 1):  
        if compuesto[candidato]:  
            continue  
  
        primos.append(candidato)  
  
        for multiplo in range(candidato * candidato, n + 1, candidato):  
            compuesto[multiplo] = True  
  
    return primos  
  
print(criba(25))
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23]
```

Léela contra la fila de arriba. `compuesto[k]` es el tachón del número k . El ciclo de afuera va recorriendo los candidatos; si uno viene tachado, `continue` salta a la vuelta siguiente. Si no viene tachado es primo, se anota, y el ciclo de adentro tacha sus múltiplos.

Dos detalles que pagan la pena de leerlos despacio. El ciclo de adentro empieza en `candidato * candidato`, no en `candidato * 2`, porque cualquier múltiplo menor que el cuadrado ya tiene un factor más chico que lo tachó antes: cuando llegamos al 5, el 10 y el 15 y el 20 ya están tachados por el 2 y por el 3. Y el paso del `range` es `candidato`, que es lo que hace que el ciclo salte de múltiplo en múltiplo sin tener que comprobar nada.

Los dos programas, cronometrados

El mismo trabajo hecho por los dos métodos: contar los primos que hay hasta un millón.

```
import math  
import time  
  
def es_primo(n):  
    if n < 2:  
        return False
```

```

    if n == 2:
        return True

    if n % 2 == 0:
        return False

    for d in range(3, int(math.sqrt(n)) + 1, 2):
        if n % d == 0:
            return False

    return True

def contar_primos(hasta):
    total = 0

    for k in range(2, hasta + 1):
        if es_primo(k):
            total += 1

    return total

inicio = time.perf_counter()
print(contar_primos(1000000), "primos, uno por uno")
print(f"{time.perf_counter() - inicio:.2f} s")

inicio = time.perf_counter()
print(len(criba(1000000)), "primos, tachando")
print(f"{time.perf_counter() - inicio:.2f} s")

```

```

78498 primos, uno por uno
1.78 s
78498 primos, tachando
0.12 s

```

Segundos contra décimas de segundo, y las dos respuestas son la misma. Pero los segundos no son un buen instrumento: dependen de la máquina, de lo que esté haciendo el sistema operativo y de la versión de Python. Corre ese bloque dos veces y te van a salir números distintos.

Lo que no cambia es **cuántas operaciones hace cada método**. El de uno en uno divide, así que se cuentan sus divisiones. La criba tacha, así que se cuentan sus tachones. Esos dos números son exactos y siempre los mismos:

```

import math

def divisiones_hasta(n):
    """Cuenta las divisiones que hace el método de uno en uno para llegar a n."""
    total = 0

    for k in range(2, n + 1):
        for d in range(2, int(math.sqrt(k)) + 1):
            total += 1

            if k % d == 0:
                break

    return total

def tachones_hasta(n):
    """Cuenta los tachones que hace la criba para llegar a n."""
    compuesto = [False] * (n + 1)
    total = 0

    for candidato in range(2, n + 1):
        if compuesto[candidato]:

```

```

        continue

    for multiplo in range(candidato * candidato, n + 1, candidato):
        compuesto[multiplo] = True
        total += 1

    return total

for tope in [100000, 300000, 900000]:
    print(f"hasta {tope:>7}: {divisiones_hasta(tope):>10} divisiones"
          f"    contra {tachones_hasta(tope):>8} tachones")

```

hasta	100000:	2745694 divisiones	contra	193078 tachones
hasta	300000:	12582930 divisiones	contra	607825 tachones
hasta	900000:	58461003 divisiones	contra	1902617 tachones

Mira esa tabla por columnas y no por filas. Cada línea multiplica el tope por tres. Los tachones de la criba también se multiplican por tres, más o menos: hacer el triple de trabajo para resolver el triple de números es lo mejor que se puede esperar. Las divisiones del otro método se multiplican por casi cinco.

Ahí está la diferencia, y no es que un programa sea diez veces más lento que el otro. Es que **la distancia entre los dos se ensancha cada vez que crece n**. Un factor de diez se paga comprando una computadora mejor. Esto no.

Eso se llama **orden de crecimiento** y es el tema de la conferencia siete, donde esta tabla se vuelve a hacer con nombres y con fórmulas. Por ahora quédate con la razón intuitiva de por qué la criba gana. El método de uno en uno empieza de cero con cada número y no aprende nada de los que ya resolvió: para decidir si 1009 es primo no usa absolutamente nada de lo que averiguó sobre 1008. La criba hace lo contrario. Cada primo que encuentra lo gasta inmediatamente en descartar de golpe a muchos otros, así que cada respuesta que consigue le abarata las siguientes.

Dejar de recalcular y empezar a recordar es la idea de la última conferencia del semestre.

9. Los docstrings

Ese texto entre comillas triples en la primera línea del cuerpo, que llevamos usando toda la clase, se llama **docstring**, y es la forma normal de decir qué hace una función. No es un comentario: Python lo guarda y lo puede mostrar.

```

def es_primo(n):
    """Dice si n es primo."""
    return n == 2

print(es_primo.__doc__)

```

```
Dice si n es primo.
```

La misma función `help` que puedes usar en la consola con `help(print)` lee esos docstrings. Escríbelos en una línea y en presente: «devuelve», «dice», «cuenta».

Y úsalos como prueba. Si no te sale un docstring de una sola frase, la función hace más de una cosa y hay dos funciones ahí dentro.

10. Resumen

- def define una función; los paréntesis la llaman. Definir no ejecuta.
- Los **parámetros** están en la definición, los **argumentos** en la llamada, y se emparejan por posición.
- return devuelve un valor y termina la función. Una función con return es una expresión y se puede usar dentro de otra cuenta.
- Un return dentro de un ciclo sale del ciclo y de la función a la vez. No necesita break.
- Una función que calcula devuelve; imprimir es trabajo del programa principal.
- Una función sin return devuelve None.
- Las variables de dentro no se ven desde fuera. Desde dentro se puede leer una global, pero asignarle crea una local y da error. Los valores entran por parámetros y salen por return.
- Los parámetros pueden traer valor por defecto, siempre al final, y en la llamada se pueden nombrar.
- Descomponer es buscar los pedazos que se explican en una frase. Esa frase es el nombre de la función, y el mejor sitio para escribirla es el docstring.
- Una función es un contrato. Mejorarla por dentro mejora gratis a todos los que la llaman, y eso es lo que hicimos cuatro veces con es_primo.

Ejercicios

1. Escribe mcd(a, b), el máximo común divisor, con el algoritmo de Euclides: el mcd de a y b es el mcd de b y el resto de dividir a entre b, hasta que el resto sea cero. Con ella escribe mcm(a, b) en una sola línea.
2. Escribe factorial(n) y úsala para escribir combinaciones(n, k), que calcula de cuántas maneras se pueden escoger k elementos de un conjunto de n. La fórmula es $\binom{n}{k} = \frac{n!}{k! (n-k)!}$. Comprueba que combinaciones(5, 2) da 10.
3. Escribe suma_de_divisores(n), que devuelve la suma de los divisores propios de n (los que son menores que n). Con ella escribe es_perfecto(n) en una línea, y con esa imprime todos los números perfectos menores que 10000. Compara este programa con el que escribiste la clase pasada para lo mismo.
4. Escribe longitud_collatz(n), que devuelve cuántos pasos tarda la secuencia de Collatz de la clase pasada en llegar a 1. Úsala para encontrar el número menor que 1000 con la secuencia más larga.
5. Los **primos gemelos** son parejas de primos que difieren en 2, como 11 y 13, o 41 y 43. Escribe contar_gemelos(n) que cuente cuántas parejas hay hasta n. Hazlo primero con es_primo y después con criba, y cronometra las dos.
6. Cuenta los primos menores que un millón con la criba, y después prueba con diez millones y con cien millones. En algún punto el programa deja de ser lento y empieza a ser imposible. Di en qué punto y por qué; la respuesta no tiene que ver con el tiempo.
7. Escribe dibujar_triangulo(altura, caracter="*") que imprima un triángulo. Con dibujar_triangulo(4) debe salir:

```
*
**
***
****
```

Añádele un parámetro invertido=False que lo dibuje al revés cuando valga True.

8. Sin ejecutarlos, di qué imprime cada uno de estos tres programas. Después compruébalo.

(a)

```
def f(x):  
    x = x + 1  
    return x  
  
y = 5  
f(y)  
print(y)
```

(b)

```
def g(n):  
    if n > 1:  
        return "puede ser primo"  
    print("seguro que no")  
  
print(g(5))  
print(g(1))
```

(c)

```
def h(a, b=2, c=3):  
    return a * 100 + b * 10 + c  
  
print(h(1))  
print(h(1, 5))  
print(h(1, c=5))
```